Running Review Doc for Math 128A

Ethan Ebbighausen

March 16, 2026

Contents

1 Week 9: (Current) **1**

2 Week 8: Divided Differences, Hermite Interpolation, Splines **1**
2.1 Divided Differences . . . 1
2.2 Hermite Interpolation . . . 2
2.3 Splines . . . 3

3 Week 7: Matrix Norms (4.4) and Polynomial Interpolation (6.1) **3**
3.1 Matrix Norms . . . 3
3.2 Polynomial Interpolation . . . 4

4 Week 6.5: Midterm Review **5**

5 Week 6: Gaussian Elimination (4.3) **6**

6 Week 5: Computing roots of polynomials (3.5), Matrix Algebra and Factorizations (4.1-4.2) **8**
6.1 Polynomial Roots . . . 8
6.2 Matrix Algebra and Factorizations . . . 10

7 Week 4: Fixed Points and Functional Iteration (3.4) **11**
7.1 Aitken's Extrapolation Formula . . . 12

8 Week 3: 3 Root finding Algorithms (3.1-3.3) **12**
8.1 Bisection Method . . . 12
8.2 Newton's Method . . . 12
8.3 Alternatives to Newton's method . . . 13

9 Week 2: Floating Point Numbers and Machine Numbers (Ch. 2) **13**
9.1 Floating Point Numbers . . . 13
9.2 Loss of Significance, Subtraction of Nearby Quantities, Accuracy Theorem . . . 14
9.3 Stability . . . 14

10 Week 1: Calculus Review (1) **14**

This document gives a running review of our Math 128A course, taught in Spring 2026 by Professor Ryan A. Hass using Kincaid and Cheney's book. This review is not comprehensive, and is just to give an overview of the topics.

1 Week 9: (Current)

2 Week 8: Divided Differences, Hermite Interpolation, Splines

We're continuing the track of approximating functions. Last week, we looked at a rigid structure of interpolation and looked at different representations of the same polynomial. We assumed we had $n+1$ distinct nodes $x_0, ..., x_n$ and data points $y_0, ..., y_n$ to interpolate from. We could construct the interpolatant by using the basis of polynomials $1, x, x^2, ..., x^n$, but this is computationally inefficient, so we used Newton's form and Lagrange's form. This week, we formalize these computations a bit more and make more general fittings of the function.

2.1 Divided Differences

Recall that in Newton's form, we pivoted to the basis $q_j(x) = \prod_{k=0}^{j-1}(x - x_k)$. We then had $f(x_i) = \sum c_j q_j(x_i)$ to solve for the coefficients, which led to a system of equations with coefficient matrix A so $a_{ij} = q_j(x_i)$, which was lower triangular by the construction of the q_j. The system looks like

$$Ac = \begin{bmatrix} f(x_0) \\ f(x_1) \\ \vdots \\ f(x_n) \end{bmatrix}$$

and we know how to solve it quickly with forward substitution. The relevant values are then what we call **divided differences**: $f[x_0] = f(x_0)$ and $f[x_0, x_1] = \frac{f(x_1)-f(x_0)}{x_1-x_0}$,...,$f[x_0, ..., x_k] = c_k$. This emphasizes that the coefficient c_k only depends on $x_0, ..., x_k$, and really just gives a new name to an object we already knew about. We can then say quite a bit about these, like that $f[x_0, ..., x_k]$ is the leading coefficient of the interpolating polynomial of f over $x_0, ..., x_k$ (show this using the formula). In other words,

$$p(x) = \sum_{k=0}^{n} f[x_0, ..., x_k] \prod_{i=0}^{k-1}(x - x_i)$$

is the interpolated polynomial. If we change the values inside the square brackets, this amounts to changing the system involved.

Using this, we have a general form for computing the interpolant coefficients

Higher-Order Divided Differences:

$$f[x_0, ..., x_k] = \frac{f[x_1, ..., x_k] - f[x_0, ..., x_{k-1}]}{x_k - x_0}$$

(notice that these start at different values!)

Proof: Let $p_n(x)$ be the polynomial interpolating f at $x_0, ..., x_n$, and let q_n denote that interpolating from $x_1, ..., x_n$. Then,

$$p_k(x) = q_k(x) + \frac{x - x_n}{x_n - x_0}[q_k(x) - p_{k-1}(x)]$$

which amounts to evaluating both sides at the points $x_0, ..., x_k$ to verify (since they are degree k polynomials, this is enough). Tracking the leading coefficient, we are done. *QED*

We may then use this to compute the $f[x_0, ..., x_k]$ coefficients in practice by taking simple differences of the $f[x_i, ..., x_j]$. See page 332 of the ext for pseudocode.

Notice also that if π is some permutation of $n+1$-tuples, e.g. $\pi(x, y) = (y, x)$ in the 2-tuple case, $f[x_0, ..., x_n] = f[(\pi(x_0, ..., x_n)]$ since these are the leading coefficient of the same interpolating polynomial.

Thus, we have some freedom in ordering things nicely. We also have the same error bounds as before written more nicely: if p is the interpolating polynomial of f at $x_0, ..., x_n$, then for $t \neq x_j$,

$$f(t) - p(t) = f[x_0, ..., x_n, t] \prod_{j=0}^{n} (t - x_j)$$

and, combining this with our MVT-form error gives for $f \in C^n$, $x_0 < ... < x_n$, $f[x_0, ..., x_n] = \frac{1}{n!} f^{(n)}(\xi)$ for some $\xi \in (x_0, x_n)$.

There is another formula for divided differences called the Hermite-Genocchi formula that relies on integration, but we won't focus on it here.

2.2 Hermite Interpolation

The name Hermite is pronounced "Air-meet". Hermite looked at a separate, orthonormal basis for polynomials that is quite useful in Fourier series computations and also simplifies our work here in some cases.

Instead of just having (x_j, y_j), imagine we have (x_j, y_j, y_j') where the second data point is for f', and we want to fit a polynomial passing through the data whose derivative passes through the primed-data. We can fit this, but must increase our degree:

Theorem on Hermite Interpolation: Let $x_0 < x_1 < ... < x_n$. For any $y_0, ..., y_n, y_0', ..., y_n' \in \mathbb{R}$, we have a unique polynomial p of degree at-most $2n + 1$ such that $p(x_j) = y_j$ and $p'(x_j) = y_j'$.

The proof is just to create a system of $2n + 2$ equations and notice it is overdetermined. Hermite interpolation refers to both this setting and also fitting higher-order derivatives. We can connect this to the previous section– let $f \in C^1(\mathbb{R})$. Then, define

$$f[x_0, x_0] = \lim_{x \to x_0} \frac{f(x) - f(x_0)}{x - x_0} = f'(x_0)$$

We think about this as the leading coefficient of the polynomial interpolating f twice at x_0, in the Hermite sense. This would look like $p(x) = f[x_0] + f[x_0, x_0](x - x_0)$. We can clearly extend to coefficients like $f[x_0, x_0, x_1]$ under this interpretation. We further define $f[x_0, x_0, ..., x_0] = \frac{1}{n!} f^{(n)}(x_0)$ to extend this to hgiher repetitions. This allows us to create a recursion as before:

Let $x_0 \leq x_1 \leq ... \leq x_n$. Then,

$$f[x_0, x_1, ..., x_n] = \begin{cases} \frac{f[x_1, ..., x_n] - f[x_0, ..., x_n - 1]}{x_n - x_0} & x_n \neq x_0 \\ \frac{1}{n!} f^{(n)}(x_0) & x_n = x_0 \end{cases}$$

[Write out what this means in the interpolation table]

Example: Let's assume we have $f(0) = 1$, $f'(0) = 1$ and $f(1) = 3$. We can interpolate at $0, 0, 1$ as follows. First, $f[x_0] = 1$, $f[x_0, x_0] = f'(x_0) = 1$, and $f[x_0, x_1] = \frac{3-1}{1-0} = 2$. Then, $f[x_0, x_0, x_1] = \frac{2-1}{1-0} = 1$. Hence, we would fit $p(x) = f[x_0] + f[x_0, x_0](x - x_0) + f[x_0, x_0, x_1](x - x_0)^2 = 1 + (x) + 1(x^2)$.

Notice that we may overlap our taylor polynomials. If $f \in C^n$ and p is a polynomial of degree at-most n so $p^k(x_0) = f^k(x_0)$ for all $k \leq n$, then p is the Taylor polynomial of f of degree n centered at x_0. Lastly, we have a similar error bound to the other cases:

Hermite Interpolation Error Estimate: Let $x_0, ..., x_n$ be distinct nodes in $[a, b]$ and let $f \in C^{2n+2}([a, b])$. If p is the polynomial of degree at-most $2n + 1$ such that $p(x_i) = f(x_i)$, $p'(x_i) = f'(x_i)$ for $0 \leq i \leq n$, then to each x in $[a, b]$ there corresponds $\xi \in (a, b)$ such that

$$f(x) - p(x) = \frac{f^{(2n+2)}(\xi)}{(2n + 2)!} \prod_{i=0}^{n} (x - x_i)^2$$

2.3 Splines

A spline function is a piecewise polynomial function. Suppose we have $n+1$ points $t_0 < t_1 < ... < t_n$ (we call these points "knots"), and that we have some integer $k \geq 0$. A spline function of degree k with these knots is a function S so that on each $[t_{i-1}, t_i)$, S is a polynomial of degree at-most k which has a continuous $(k-1)$st derivative on $[t_0, t_n]$. This allows us to keep small degree while fitting functions that do not grow like polynomials, such as $\sin(x)$. Cubic splines are commonly used in practice, and we will construct them here.

Assume we have a table of values $t(i, y_i)$ for $i = 0, ..., n$. We will use these both as knots and as interpolation nodes. Let S_i denote the cubic polynomial representing S on $[t_i, t_{i+1}]$. We should have $S_{i-1}(t_i) = y_i = S_i(t_i)$. We must also have $S'_{i-1}(t_i) = S'_i(t_i)$ and $S''_{i-1}(t_i) = S''_i(t_i)$ for the continuity of the derivative. These three types of conditions give $4n-2$ equations. However, we have $4n$ coefficients that we are treating as variables to solve for (4 coefficients for each of n S_i). Thus, we have two remaining degrees of freedom.

[Finish]

3 Week 7: Matrix Norms (4.4) and Polynomial Interpolation (6.1)

3.1 Matrix Norms

Recall (from 110, 104, or 54) that we may define a norm $||\cdot||$ on a vector space V, which has the propoerties that it is positive definite, homogeneous, and satisfies the triangle inequality. Our standard norm on $\mathbb{R}^n$ is the Euclidean norm $||x|| = \sqrt{\sum_{i=1}^n x_i^2}$. However, we may also define the p-norm $||x||_p = [\sum_{i=1}^n |x_i|^p]^{1/p}$, and the limiting infinity-norm
$normx_\infty = \max |x_i|$. As an exercise, check that these satisfy the norm properties. It will not be relevant for our course, but all norms on $\mathbb{R}^n$ are equivalent (via a continuity argument).

For our first definition of a norm on matrices, we view matrices as linear transformations and apply the operator norm (which you may see in many other contexts)

$$||A|| = \sup\{||Au|| \mid u \in \mathbb{R}^n,\ ||u|| = 1\}$$

where we have some defined norm on $\mathbb{R}^n$. Notice there are two different meanins og the norm bars in this statement– this usage is common, so try to use context to determine what norm is meant. Our text calls this the **subordinate matrix norm** because of this intrinstic tie. Proving that this is truly a norm also relies on the properties of the norm on $\mathbb{R}^n$ (and you should check that they hold). In the case that the underlying norm is Euclidean, this is called the spectral norm (because $||A||_2 = \max_{1 \leq ileqn} |\sigma_i|$, the maximum singular value of the matrix, which we prove later).

Example
Since $||Ix|| = ||x||$ (where I is the identity), $||I|| = 1$. In the case that our underlying norm is the Euclidean norm (which arises from an inner product), any othogonal matrix preserves distances, and also has norm 1. This includes rotation matrices.

Using the homogeneity of the norm $||Ax|| = ||x|| \cdot ||A\frac{x}{||x||}|| \leq ||A||||x||$, an important property of this norm. Expanding similarly, we see that $||ABx|| \leq ||A||||B||||x||$ implies $||AB|| \leq ||A||||B||$.

Another commonly used matrix norm is

$$||A||_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{i,j}|$$

which takes the largest row sum. This is the matrix norm subordinate to the infinity norm defined above (prove this and check norm properties).

Suppose we have a matrix equation $Ax = b$. We now have a way to understand "moving this equation slightly": if A is perturbed slightly, A^{-1} is perturbed to some new matrix B and the resulting $x = A^{-1}b$ will

be pertubred to $\tilde{x} = Bb$ as well. We can measure this error:

$$||x - \tilde{x}|| = ||x - BAx|| \leq ||I - BA||||x||$$

or

$$\frac{||x - \tilde{x}||}{||x||} \leq ||I - BA||$$

giving a realtive error bound. We may aslo measure the residual error instead:

$$||x - \tilde{x}|| \leq ||A^{-1}||||b - A\tilde{x}|| = ||A^{-1}||||Ax||||b - \tilde{b}||||b||^{-1}$$

or

$$\frac{||x - \tilde{x}||}{||x||} \leq \kappa(A)\frac{||b - \tilde{b}||}{||b||}$$

where $\kappa(A) = ||A^{-1}||||A||$ is called the condition number of the matrix A. Let $e = x - \tilde{x}$ and $r = b - \tilde{b}$. We can get simple bounds from the above examples:

$$\frac{1}{\kappa(A)}\frac{||r||}{||b||} \leq \frac{||e||}{||x||} \leq \kappa(A)\frac{||r||}{||b||}$$

this shows how large condition numbers can lead to big errors, and so we expect a loss of precision in solving $Ax = b$ numerically. The whole of this study gives us an easy number to compute to encapsulate this.

3.2 Polynomial Interpolation

Assume you have some sort of function that you can get outputs for, but you have no formula or easy way to represent. How might you work with said function? One solution would be to compute the function at a bunch of points, connect them with lines, and use those lines to represent the function itself. If the function appears like it should curve a lot, maybe we could use a qudratic of polynomial instead. This is the idea of polynomial interpolation. We have some data points (x_i, y_i) for $i = 0, 1, ..., n$, and we wish to interpolate this data into a function, so we fit some polynomials to pass through them.

For n data points, we can fit a unique degree-n polynomial to pass through all of them. One way to do this is to make polynomials of increasing degree as we fit more data. For example,

$$p_k(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + ... + c_k(x - x_0)...(x - x_{k-1})$$

for each $0 \leq k \leq n$. In each step, we add 1 term with coefficient to account for the new y value. Hence, $c_0 = y_0$, $c_1 = \frac{y_1 - c_0}{x_1 - x_0}$.

These polynomials are called interpolation polynomials in Newton's form. We may compute values of the polynomial efficiently using Horner's method, as we already discussed, so this is an easy way to represent functions and bridge the gap in data. In general, we also can easily compute

$$c_k = \frac{y_k - p_{k-1}(x_k)}{(x_k - x_0)...(x_k - x_{k-1})}$$

An alternative method is to represent the polynomial as

$$p(x) = \sum_{k=0}^{n} y_k l_k(x)$$

where the $l_k(x)$ depend on the nodes x_i but not the ordinates y_j. In this case, we have

$$l_k(x) = \prod_{\substack{j=1 \\ j \neq k}}^{n} \frac{x - x_j}{x_k - x_j}$$

known as the cardinal functions or Lagrange form of interpolating polynomials. This will always agree with the polynomial computed in Newton's form, and is simply a different representation that makes evaulation a bit less expensive .

How good is this approximation?

Theorem on Interpolation Error Let f be a $C^{n+1}([a,b])$ function and let p be the polynomial of degree at-most n that interpolates f at the $n+1$ distinct points $x_0, x_1, ..., x_n$ in $[a,b]$. To each $x \in [a,b]$, these is a $\xi_x \in (a,b)$ so

$$f(x) - p(x) = \frac{1}{(n+1)!} f^{(n+1)}(\xi_x) \prod_{i=0}^{n} (x - x_i)$$

Proof We fix x, then set $w(t) = \prod_{i=0}^{n}(t - x_i)$ and $\phi(t) = f(t) - p(t) - \frac{f(x)-p(x)}{w(x)} w(t)$. By Rolle's theorem, $\phi^{(n+1)}$ has at least one zero, ξ_x, in (a,b) and so $0 = \phi^{(n+1}(\xi_x) = f^{(n+1)}(\xi_x) - (n+1)! \frac{f(x)-p(x)}{w(x)}$. *QED*

This is where the professor stopped, but we can expand a tad bit more. First, we are assuming that the nodes are given to us, but what if we can choose them? If we choose the nodes in a special way, maybe we can get a better approximation to the function. This was investigated by Chebyshev, leading to the Chebyshev polynomials. They are defined recursively: $T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x)$ with $T_0(x) = 1$ and $T_1(x) = x$. If we allow the nodes x_i to be the roots of the Chebyshev polynomial T_{n+1} (and consider only $|x| \leq 1$), then we get an approximation error $|f(x) - p(x)| \leq \frac{1}{2^n(n+1)!} \max_{|t| \leq 1} |f^{(n+1)}(t)|$, an improvement over the last bound.

Now, we have some cool error bounds, but why should we trust interpolation at all? If we focus on a closed interval $[a,b]$, or really any compact set, the Stone-Weierstrass Theorem says that polynomials are dense in $C([a,b])$ under the uniform norm. This is to say we can approximate a function arbitrarily well. There are results that allow us to get an appropriate polynomial approximating f to be an interpolant, if the nodes are selected carefully. However, for poorly selected nodes, we can always break this uniform approximation, so we do need to be careful (see Faber's theorem).

4 Week 6.5: Midterm Review

The midterm covers chapters 1-3, so the calculus review, floating point numbers and the involved error, root finding techniques, and functional iteration. This whole document is a review document, bit here are some problems pulled from the text that I will use as practice during our review session in discussion. You should understand the pratice midterm and prioritize that over these problems, however.

1.) State Taylor's theorem with Lagrange Error. How does the Taylor polynomial approximate the function in big and little O notation?

1.1) Compute $ln(1.5)$ with accuracy at least 1/10 (you can write a sum and not evaluate it).

You should use the Taylor series for natural log about $x = 1$ and take a large enough sum so the error term may be bounded above by 1/10th. The 10-term sum is plenty for this

1.2) Show $1 + x \leq e^x$ for all $x \geq 0$. *Use the Taylor series for e^x and look at the first two terms*

2.) What is the larges k so $\arctan(x) = x + O(x^k)$.

The Taylor series for arctan *shows this is $k = 3$*

2.1) Is it true that $\frac{1}{n \ln(n)} = o(1/n)$

Yes, this is a direct definition-checking problem

2.2) Let $a_n \to 0$ and $\lambda > 1$. Show $\sum_{k=0}^{n} a_k \lambda^k = o(\lambda^n)$.

Compute $\lambda^{-n} \sum_{k=0}^{n} a_k \lambda^k = a_n + a_{n-1}\lambda^{-1} + ... + a_0 \lambda^{-n}$. As $n \to \infty$, terms like a_n, a_{n-1} approach 0 because the sequence does, and terms like $a_0 \lambda^{-n}$ approach 0 because $\lambda > 1$ and n is large

3.) Let $x = 2^{12} + 2^{-12}$. Find a machine number representation x^* for x (in Marc-32). What is the relative error between x and x^*?

You should see that this is evenly between two machine numbers, so either works, and that the relative error is 2^{-24}

3.1) If we compute the product of 3 numbers x_1, x_2, x_3 under floating point arithmetic, what is the maximum relative error between $fl(x_1x_2x_3)$ and $x_1x_2x_3$?

We must view the product as $fl(fl(fl(x_1)fl(x_2))fl(x_3)$. *We have* $fl(x_i) = x_i^* = x_i(1+\delta_i)$ *where* $|\delta_i| < 2^{-24}$ *in Marc-32. Then,* $fl(fl(x_1)fl(x_2)) = (x_1)(1+\delta_1)(x_2)(1+\delta_2)(1+\epsilon_1)$, *where the extra error comes from the float of the final product. Repeating this again gives* $x_1x_2x_3(1+\delta_1)(1+\delta_2)(1+\delta_3)(1+\epsilon_1)(1+\epsilon_2)$. *Multiplying these gives a relative error of order approximately* $5 \cdot 2^{-24}$.

3.2) Let $n\epsilon < 0.01$. Find an upper bound for $(1+\epsilon)^n - 1$ (not floating point!). One bound is 0.01006.

5 Week 6: Gaussian Elimination (4.3)

Linear algebra is extremely important to numerical problems. In last week's material, we went into a whole section on breaking down matrices via several factorizations just to make solving them easier in special cases. How can we compute these factorizations easily? We use a similar process to row reduction, but make the choices more consistent so that we can automate the process. Our goal will be Doolittle's factorization.

Let $A \in M(n)$. To perform Gaussian elimination on A, we perform a sequence of $n-1$ steps resulting in a sequence of matrices $A = A^{(1)} \to ... \to A^{(n)}$ where the fimal matrix is upper triangular. We will have

$$A^{(k)} = \begin{bmatrix} U^{(k-1)} & B^{(k)} \\ 0 & C^{(k)} \end{bmatrix}$$

where $U^{(k-1)}$ is upper triangular. In other words, we perform row reduction sequentially so that the rows above the row in the step of interest remain unchanged. How is this possible? In general, it may not be, and this is where our previous theorems on LU factorization make more sense. Recall that, if a matrix has nonsingular principal minors, it has a LU factorization.

The algorithm proceeds as follows. Assume we have matrix $A^{(k)} = (a_{i,j}^k)$ and want to form $A^{(k+1)}$. For the first k rows of $A^{(k)}$, we leave them be. Next, if i and j are both at least $k+1$, we reset values by subtracting off multiples of the pivot rows above:

$$a_{i,j}^{k+1} = a_{i,j}^k - \left(a_{i,k}^k / a_{k,k}^k\right) a_{k,j}^k$$

the remaining entries are all zeroes. For example, on the matrix

$$\begin{bmatrix} 6 & -2 & 2 \\ 12 & -8 & 6 \\ 3 & -13 & 9 \end{bmatrix}$$

we perform the iteration

$$\begin{bmatrix} 6 & -2 & 2 \\ 12 & -8 & 6 \\ 3 & -13 & 9 \end{bmatrix} \to \begin{bmatrix} 6 & -2 & 2 \\ 0 & -4 & 2 \\ 0 & -12 & 8 \end{bmatrix} \to \begin{bmatrix} 6 & -2 & 2 \\ 0 & -4 & 2 \\ 0 & 0 & 2 \end{bmatrix}$$

If all the pivots $a_{k,k}^k$ of this process are nonzero, then this produces $U = A^{(n)}$ and we collect the row operations for L:

$$l_{i,k} = \begin{cases} a_{i,k}^k / a_{k,k}^k & i \geq k+1 \\ 1 & i = k \\ 0 & i \leq k-1 \end{cases}$$

When doing this by hand, it's easier to just keep track of the multipliers you used for each step (for the above, $L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1/2 & 3 & 1 \end{bmatrix}$). To prove this statement, check that for L and U formed this way, $LU = A$. This amounts to chasing definitions.

This algorithm is great when it applies, but because it assumes the matrix has good structure, we should also try to form an algorithm that can address zeroes in the pivots as well as floating point and rounding issues that arise from having small pivots (you should be able to explain why small pivots can be an issue, perhaps using the example $\begin{bmatrix} \epsilon & 1 \\ 1 & 1 \end{bmatrix}$). We call this process pivoting. You already know this process from using row reduction to get row echelon form, or perhaps reduced row echelon form to account for large pivots. In that case, we actually swap the rows, but here we will not swap them but still treat them as such and keep track of what permutation needs to be done to get an upper triangular matrix.

An algorithm for this process is scaled pivoting. First, we start with a square matrix $A \in M(n)$. To find the "large pivots", compute the scales $s_i = \max_{1 \leq k \leq n}\{|a_{i,k}|\}$ for $i = 1, ..., n$ and select the first pivot row $A^{(1)}_{p_1}$ where $|a_{p_1,1}|/s_{p_1} \geq |a_{i,1}|/s_1$ (where we at least reach the maximum relative size for the first column). We use this row to reduce the others, and log the number p_1 so the first column of our permutation matrix is e_{p_1}. Next, we repeat throughout the rest of the columns, where we exclude previous pivot rows from our maximum choices. Doing this n times returns $A^{(n)}$ and P, so $PA = LU$ where $U = PA^{(n)}$, and L again tracks our progress.

Example: For a small-scale example, consider

$$A = \begin{bmatrix} 0 & 1 & 2 \\ 1 & 1 & 1 \\ 3 & 2 & 4 \end{bmatrix}$$

Our first step will choose $p_1 = 3$ and subtract to

$$A^{(1)} = \begin{matrix} 01 & & 2 \\ 01/3 & & -1/3 \\ 32 & & 4 \end{matrix}$$

with

$$P = \begin{bmatrix} 0 & ? & ? \\ 0 & ? & ? \\ 1 & ? & ? \end{bmatrix}$$

For column 2, we exclude the third row and choose $p_2 = 1$, so

$$P = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

Gaussian elimination computes $A^{(2)}$ and $A^{(3)}$ in the obvious way.

There is another algorithm in the book that we will not focus on called complete pivoting. When the previous processes move forward th column k, they each consider $n - k + 1$ elements in the column to make pivot choices (n elements minus the previous pivot rows). Complete pivoting instead considers the $(n-k+1)^2$ elements of the submatrix formed by excluding previous pivot rows and columns, then picks either a pivot row or column.

Let the resulting permutation matrix be P, and this solve $PA = LU$. When we now go to solve $Ax = b$, we perform this process and have $PAx = Pb$, giving $LUx = Pb$. Updating $y = Pb$, we solve $Lz = y$ then $Ux = z$ to find x.

There are some cases where we may safely used Gaussian Elimination as opposed to one of these pivoting algorithms. A matrix is called diagonally dominant if

$$|a_{i,i}| > \sum_{\substack{j=1 \\ j\neq i}}^{n} |a_{i,j}|$$

for all $1 \leq i \leq n$. Gaussian elimination without pivoting preserves the diagonal dominance of a matrix, and so by our previous discussion it gives and LU factorization where U is diagonally dominant. As U is upper triangular, this implies it has nonzero diagonal entries and that the original A was also nonsingular. Applying full or scaled row pivoting to a diagonally dominant matrix will also be the exact same process to Gaussian elimination, so this is really the special case which amounts to tracking that choice of pivots.

6 Week 5: Computing roots of polynomials (3.5), Matrix Algebra and Factorizations (4.1-4.2)

6.1 Polynomial Roots

Due to the work of Gauss, we know that a degree n polynomial has n complex roots (the Fundamental Theorem of algebra paired with polynomial division). Following our trend on root finding to replace various proceeses like rational roots, we would like to find an efficient method of computing these roots. We start with localization theorems, which tell us a range of where the roots may lie. Consider a polynomial $p(x) = a_n z^n + a_{n-1} z^{n-1} + ... + a_0$.

Localization Theorem All the roots of $p(x)$ written as above lie in $B(0, \rho) \subset \mathbb{C}$ where $\rho = 1 + |a_n|^{-1} \max_{0\leq k\leq n} |a_k|$.

Proof: Let $c = \max_{0\leq k\leq n} |a_k|$. The case $c = 0$ is immediate, so assume $c > 0$ and $\rho > 1$. For $|z| \geq \rho$, (use the reverse triangle inequality and geometric sums).

$$|p(z)| \geq |a_n z^n| - c \sum_{k=1}^{n-1} |z|^k > |a_n z^n| - c|z|^n (|z| - 1)^{-1} \geq |a_n z^n|(1 - c|a_n|^{-1}(\rho - 1)^{-1}) = 0$$

QED

This theorem implies a second:

Corollary: If all the roots of $z^n p(1/z)$ are in the disk $B(0, \rho)$, then all the non-zero roots of p are outside $B(0, \rho^{-1})$.

Proof: The polynomial $z^n p(1/z)$ is $s(z) = a_n + a_{n-1} z + ... + a_0 z^n$. Then, $p(z_0) = 0$ is equivalent to $s(1/z_0) = 0$. Since the roots of s have $|z| \leq \rho$, we then have $\frac{1}{|z|} \geq \rho^{-1}$, as desired. *QED*

These results give a starting point for algorithms to take hold. We will focus on Newton's method, but the text also covers Bairstow's method and Laguerre iteration. For Newton's method, we want an easy way to comput the polynomial, especially centered at different points or in the nested multiplication form from the first week. For this, we use Horner's method.

Starting with a polynomial $p(z)$ of the above form, we perform an artificial polynomial division to reduce the degree of the polynomial. Since $p(z) - p(z_0)$ has a root at z_0, we may form $q(z) = \frac{p(z)-p(z_0)}{z-z_0}$ a degree $n-1$-polynomial such that $p(z) = p(z_0) + q(z)(z - z_0)$. Horner's method tells us how to compute this $q(z)$ and $p(z_0)$. Let $q(z) = b_{n-1} z^{n-1} + ... + b_0$. Then, using the product relationship $p(z) = p(z_0) + q(z)(z - z_0)$, $a_n = b_{n-1}$, and $a_j + z_0 b_j = b_{j-1}$ for $0 \leq j \leq n-1$. Rewriting this relationship, we have $b_k = a_{k+1} + z_0 a_{k+2}$. We may compute this all the way down to b_0, and if we put in $k = -1$, the value b_{-1} we solve for is actually $p(z_0)$.

One way this helps our algorithms on the computer side is that it allows us to compute $p'(z_0)$ without needed to use symbolic logic in the comuter (i.e. without using the power rule). Using a taylor series on p, we may see

$$p(z) = c_n(z - z_0)^n + ... + c_1(z - z_0) + c_0$$

where $c_1 = p'(z_0)$. Applying Horner's method at z_0 gives $p(z) = (z - z_0)q(z) + c_0$. It then follows that $q(z_0) = p'(z_0)$ from the product rule. Another way to see this is to write

$$p(z) = c_0 + (z - z_0)(c_1 + (z - z_0)(c_2 + ... + (z - z_0)c_n)...)$$

where, aligning with Horner's relationship, $q(z) = (c_1 + (z - z_0)(c_2 + ... + (z - z_0)c_n)...)$. Repeating Horner's method on q then gives the higher Taylor coefficients of p: for $p(z) = (z - z_0)q_0(z) + p(z_0)$ and $q_i(z) = q_{i+1}(z)(z - z_0) + q_i(z_0)$; then $q_k(z_0) = p^{(k)}(z_0)/k!$ and these values are the last value produced in each step of Horner's method.

We may now work on finding roots. In the special case z_0 is a root, our formula is simply $q(z) = \frac{p(z)}{z-z_0}$, so this division merely decreases the degree of p without any added complication. We may apply Newton's method to $p(z)$, computing its values and its derivative's values by Horner's method, then have $p(z) = (z - z_0)q(z)$ and repeat this idea on $q(z)$. In a finite number of steps, we will have fully factored $p(z)$.

The localization methods give us a good starting area, and we may use polynomial coefficients or estimates to get an idea on a radius that implies convergence for Newton's method. We also have the following:

Theorem on Successive Newton Estimates (Bodewig 1946) Let x_k and x_{k+1} be two successive iterates when NEwton's method is applied to a degree n polynomial p. Then, there is a root of p within distance $n|x_k - x_{k+1}|$ of x_k in the complex plane.

Proof Let $r_1, ..., r_n$ be the roots of p. Then, $p(z) = c\prod_{j=1}^{n}(z - r_j)$, and

$$p'(z) = c\sum_{i=1}^{n}\prod_{j\neq i}(z - r_j) = \sum_{i=1}^{n}\frac{p(z)}{z - r_i}$$

We claim that for and z, there is some index j so $|z - r_j| \leq n|p(z)/p'(z)|$. If this were to fail, we would have

$$|z - r_j|^{-1} < \frac{1}{n}|p'(z)/p(z)| = \frac{1}{n}|\sum(z - r_i)^{-1} \leq \frac{1}{n}\sum|z - r_i|^{-1}$$

Since the number on the right is an average, this is impossible. Notice now that $n|x_k - x_{k+1}| = n|-\frac{p(x_k)}{p'(x_k)}|$, so that we have shown root is within this radius of x_k.

QED

This theorem may be useful for your code or HW, and gives us an idea of how to set error bounds.

6.2 Matrix Algebra and Factorizations

In Math 54, we learned several factorizations like diagonalization, LU-factorization, QR-factorization, and Singular Value decomposition. These will be useful in making code and algorithms more efficient, so we review. Let $A \in M(n, \mathbb{R})$, which means A is a $n \times n$ matrix with entries in $\mathbb{R}$. We denote $A = (a_{i,j})$ to say that the $a_{i,j}$ are the entries of A in the i, j positions. We also sometimes denote the rows of A by A_i or the columns by A_j.

Recall that we do many processes like Gaussian elimination, row reduction, and even diagonalization by elementary row operations. These operations include adding rows, interchanging rows, and multiplying rows by a nonzero constant. For each operation, there exists a matrix E so EA is the same as doing the row operation to A. These matrices E are invertible, and their inverse signifies the opposite row operation. Recall that EA and AE may be different, as multiplication is not commutative for matrices. You should

review what being an invertible matrix means, how to compute simple inverses, and how invertibility for a square matrix is the same as being injective, surjective, being the product of elementary matrices, or having nonzero determinant. There are some guiding questions on a MAth 54 review doc from my older teaching sites.

We begin with LU-factorization. Recall that L is a lower triangular matrix if all entries strictly below the diagonal are 0, or $l_{i,j} = 0$ for $i < j$. We may similarly have an upper triangular matrix U. These types of matrices are useful for numerical methods because solving $Lx = b$ or $Ux = b$ is quite easy compared to for a standard matrix. For example, consider

$$\begin{bmatrix} a & 0 \\ b & c \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \end{bmatrix}$$

Then, we have $ax_1 = d_1$, so $x_1 = d_1/a_1$. This helps us solve the second equation, $bx_1 + cx_2 = d_2$ then becomes $x_2 = c^{-1}(d_2 - bd_1a_1^{-1})$. For larger matrices, the same recursion applies to simplyifying inverting L.

A square matrix A has an LU factorization if $A = LU$ where L is lower triangular and U is upper triangular. Like many other factorizations, this is not unique: if D is an invertible diagonal matrix, and $A = LU$, then $A = (LD)(D^{-1}U)$ is another LU-factorization. When A is invertible, it is possible to show that all LU factorizations of A are related by such a relationship.

What criteria do we have to imply an LU-factorization exists? Let $B \in M(k)$ be a sub-matrix of A. The leading principal matrices of A are those B where $b_{i,j} = a_{i,j}$ for all i, j, or where B is located in the top left corner of A. If all leading principal matrices of A are invertible, then A has n LU-factorization (the proof of this is semi-long, but should be understood. I will replicate it here at a later date).

We usually compute the LU-factorization by applying row operations to A to get an upper triangular matrix U, then collecting the elementary matrices to what should be a lower triangular matrix for L (if an LU factorization even exists). For example, consider the matrix

$$\begin{bmatrix} 8 & 4 \\ 4 & 1 \end{bmatrix}$$

By subtracting the half of the first row from the second, we obtain an upper triangular matrix

$$U = \begin{bmatrix} 8 & 4 \\ 0 & -1 \end{bmatrix}$$

and this elemetary row operation is encoded by $\begin{bmatrix} 1 & 0 \\ -0.5 & 1 \end{bmatrix}$. The inverse of this operation is $L = \begin{bmatrix} 1 & 0 \\ 0.5 & 1 \end{bmatrix}$, and so we have $A = LU$ is an LU-factorization.

A more foolproof way to compute it theoeretically would be to set up systems of equations, but such a method includes exactly the type of problem we are trying to solve. Simplifying this system leads to the special case of Doolittle's factorization, an LU-factorization with $l_{i,i} = 1$ for $i = 1, ..., n$ (depending on your reference, these may just be nonzero, which is equivalent by the comments above about diagonal matrices). We may then recursively solve for the values of L and U using the equations

$$u_{k,j} = a_{k,j} - \sum_{s=1}^{k-1} l_{k,s} u_{s,j}$$
$$l_{i,k} = \left[a_{i,k} - \sum_{s=1}^{k-1} l_{i,s} u_{s,k} \right] / u_{k,k}$$

Try multiplying out abstract matrices in the 3×3 case to make sense of these equations. You should notice things like the first row of U and the first row of A being the same, and move from there.

Similarly, Crout factorization is an LU-factorization with $u_{i,i} = 1$ for all i, and also has a recursive algorithm for computing the entries of L and U. We also have a decomposition that looks to LU-factorization as orthogonal diagonalization does to diagonalization. Cholesky's factorization is an LU-factorization where $U = L^T$, or $A = LL^T$. Such a factorization exists and is unique if A is a symmetric and positive definite matrix, but may exist in much broader cases.

7 Week 4: Fixed Points and Functional Iteration (3.4)

The generalizations of Newton's method may be posed as a type of function iteration. If we define $F(x) = x - \frac{f(x)}{f'(x)}$, where f is the function we are trying to find a root of, then Newton's method becomes $F(x_n) = x_{n+1}$. This is a useful situation because there is a strong theorem in analysis that helps us find limits in such cases, called the contraction mapping theorem.

This theorem supposes that we have a non-empty complete metric space X, and defines a function $F : X \to X$ to be a contraction if $|F(x) - F(y)| \leq c|x - y|$ where $c \in (0, 1)$. Under this condition, it poses that we have a unique fixed point x^*, i.e. a point where $F(x^*) = x^*$, and the proof of the theorem shows that we may reach such x^* by taking any point y and $F^n(x) = F(F(...F(x)...))$ converges to x^* as $n \to \infty$.

In the context of the Secant method or Newton's method, we then converge to a root if we can pose appropriate F as a contraction. The MVT is often helpful in showing appropriate functions are contractions on the right spaces.

Finally, we care about the rate of convergence. Given F and $x_{n+1} = F(x_n)$, where F is a contraction with fixed point x^*, we trace $e_n = x_n - x^*$. Under suitable assumptions,

$$e_{n+1} = F(x_n) - F(s) = F(s + e_n) - F(s) = e_n F'(s) + \frac{1}{2} e_n^2 F''(s) + ... + \frac{1}{(q-1)!} e_n^{q-1} F^{(q-1)}(s) + \frac{1}{q!} e_n^q F^{(q)}(\zeta_n)$$

so that if s is an order -q root of F, then $e_{n+1} = \frac{1}{q!} e_n^q F^{(q)}(\zeta_n)$ and we have q-order convergence. Try checking the convergence for Newton's method and the Secant method.

Example: Determine the order of convergence of the sequence $x_n = n^{-1/2}$. We should expect linear convergence from normal limit arguments, but let us check using this method. Squaring the terms, we see $x_n^2 = n$, so that $x_n^{-2} + 1 = n + 1 = x_{n+1}^{-2}$. This is to say

$$F(x) = \frac{1}{\sqrt{\frac{1}{x^2} + 1}} = \frac{x}{\sqrt{1 + x^2}}$$

$$F'(x) = \frac{1 - x^2}{(x^2 + 1)^{3/2}}$$

so that $F'(0) \neq 0$. We expect linear convergence, and this may also be checked directly from the definition of the order of convergence.

7.1 Aitken's Extrapolation Formula

Let $F \in C^1(\mathbb{R})$ be a contraction with fixed point s, and $x_{n+1} = F(x_n)$ defining a sequence converging to that point. Then,

$$s - x_n \approx \frac{\lambda_n}{1 - \lambda_n}(x_n - x_{n-1})$$

$$\lambda_n = \frac{x_n - x_{n-1}}{x_{n-1} - x_{n-2}}$$

Aitken's method stipulates that to compute s, starting with x_1, we compute $x_2 = F(x_1)$ and $x_3 = F(x_2)$, then compute x_4 via the above formula. To proceed, we compute $x_n = F(x_{n-1})$ if $n \equiv 1, 2, 3 \mod 4$ and use Aitken's formula for x_{4k}.

8 Week 3: 3 Root finding Algorithms (3.1-3.3)

8.1 Bisection Method

The bisection method starts with a *continuous* function f and an interval $[a, b]$ on which we have $f(a)f(b) < 0$. This is a special assumption to guarantee that we have a root of our function f on $[a, b]$ by the Intermediate Value Theorem.

We then produce $c = \frac{a+b}{2} = a + \frac{b-a}{2}$, the midpoint of the interval. We check whether $f(a)f(c) < 0$, in which case we set $[a_1, b_1] = [a, c]$. In the case $f(a)f(c) > 0$, we know $f(a)f(b) < 0$, so we et $[a_1, b_1] = [c, b]$. Finally, if $f(c) = 0$, we terminate. We repeat this process, producing c_1 and then $[a_2, b_2]$.

This way, each step cuts the interval in half. We use c_n as out approximation of the root. If r is the root we eventually converge to, we know $|c_n - r| \leq \frac{1}{2}|b_n - a_n| \leq 2^{-n}|b - a|$.

8.2 Newton's Method

For Newton's method, we assume that we have a C^2 function f for which we want to find a root. We start with some initial value x_0. Then, we assume there exists some root r, and so we know $0 = f(r) = f(x_0 - e_0)$ where $e_0 = x_0 - r$. Expanding by Taylor's theorem, $0 = f(x_0) - e_n f'(x_0) + ...$, so we approximate $e_0 \approx f(x_0)/f'(x_0)$ and update

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

at each subsequent step.

This isn't guaranteed to converge for any starting point, but let's look at the case of convergence.

$$e_{n+1} = x_{n+1} - r = x_n - \frac{f(x_n)}{f'(x_n)} - r = e_n - \frac{f(x_n)}{f'(x_n)}$$

Thus, we use that $0 = f(x_n) - e_n f'(x_n) + e_n^2 f''(\zeta_n)/2$ from above to rewrite this as

$$e_{n+1} = \frac{e_n f'(x_n) - f(x_n)}{f'(x_n)} = \frac{1}{2}\frac{f''(\zeta_n)}{f'(x_n)} e_n^2$$

When do we see convergence? We need the $e_n \to 0$ for this, which is okay when this coefficient is bounded. This is a second place where we use the C^2 assumption. Set

$$c(\delta) = \frac{1}{2}\frac{\max_{|x-r|<\delta} |f''(x|)}{\min_{|x-r|<\delta} |f'(x)|}$$

Using continuity, take δ small enough so $\delta c(\delta) < 1$. Then, it is easy to note that if $|e_n| \leq \delta$, then $|e_{n+1}| \leq \delta$ as well and that $|e_{n+1}| \leq C|e_n|^2$, so that we have quadratic convergence when x_0 is chosen within a δ-ball of r.

8.3 Alternatives to Newton's method

In cases where we don't want to compute the derivative, several alternatives exist. The first approximation we might take is that $f'(x) \approx \frac{f(x+h)-f(x)}{h}$ for small h. Using this in Newton's method gives the Secant method,

$$x_{n+1} = x_n - \frac{f(x_n)}{(f(x_n) - f(x_{n-1}))/(x_n - x_{n-1})}$$

though several alternatives exist, including Steffenson's method (further approximation) and Halley's method (which requires the second derivative). We will explore their convergence rates through functional iteration next week.

9 Week 2: Floating Point Numbers and Machine Numbers (Ch. 2)

9.1 Floating Point Numbers

It would be amazing if we could punch numbers into computers and store exact values, but out computers only have finite memory and numbers like π that are irrational can't be stored in a finite number of base-10 digits. Furthermore, the hardware restrictions of semiconductors when computers were developed means that base-10 is not even natural to create mechanically. For these reasons, we instead store numbers in a finite string of 0s and 1s called bits, which represent a rounded version of the value. This type if storage is called floating point storage, and it can cause some issues if we aren't careful.

First, we represent numbers in scientific notation: $x = (-1)^s r \times 10^n$ where $r \in [0.1, 1)$ and n is an integer. This gives a way to store a spread of values in a consistent way. In binary, this would be of the form $x = (-1)^s q \times 2^m$. The number q is called the mantissa, and m is called the exponent. Since $q \in [1/2, 1)$, we always have a leading 1, so we actually shift the mantissa over by one digit to squeeze more precision out of the same number of bits. Since we want to store both large and very small numbers without needing an extra negative sign for the exponent (which takes extra storage), we also bias the exponent

$$x = (-1)^s 1.f \times 2^{e-127}$$

This is called a normalized floating point form, and is also the IEEE standard floating point representation.

Then, $s \in \{0, 1\}$ will always account for at most 1 bit. The other numbers f and e have lengths depending on the system. In a standard 32-bit system, f recieves 23 bits and e recieves 8 bits.

Numbers of exactly this form are called machine numbers. Some very simple decimal numbers are not machine numbers, such as 0.01. In such cases, most machines round a number to its nearest machine number (though there are other rounding systems in use). In practice, this means we compute the number x in binary to some accuracy, then truncate the expansion to get a machine number just below it, x_1. We also look at the next machine number above x_-, x_+. When the check which is smaller between $x_+ - x$ and $x - x_-$ to determine the machine number.

When arithmetic is done, the computer adds bits to make for accurate calculations. The standard for a 32-bit storage is 80 bits for internal calculations (check out the IEEE standards for more info).

Sometimes, special exponents like $e = 0$ and $e = 255$ are reserved for 0 and ∞, and the computer may return a NaN (not a number). Alternatively, arithmetic may produce numbers of exponent smaller or larger than the floating point storage allows, which are called underflow and overflow errors. There are some interesting examples of these errors causing problems in the real world.

The above process of selecting machine numbers close to our number doesn't have a consistent absolute error. Consider the machine ϵ, or the distance between machine numbers in a given range. For Marc-32 and machine numbers in $[1/2, 1)$, this ϵ is 2^{-24}. For numbers in $[1, 2)$, the ϵ is 2^{-23}. However, what we do preserve is the relative error. In Marc-32, our relative error will be at-most 2^{-24}. We may pose $fl(x) = x(1 + \delta)$ where $|\delta| \leq 2^{-24}$ to do error analyises. For example, taking the product of two floating point numbers. We have one main theorem using these:

Theorem on Relative Rounding Error in Adding: Let $x_0, x_1, ..., x_n$ be positive machine numbers in a computer whose unit roundoff error is ϵ. Then, the relative roundoff error in computing $\sum_{i=0}^{n} x_i$ in the usual way is at most $(1 + \epsilon)^n - 1 \approx n\epsilon$.

9.2 Loss of Significance, Subtraction of Nearby Quantities, Accuracy Theorem

Consider the case of $x = 2^n + 2^k$ and $y = 2^n$, so $x - y = 2^k$. However, in floating point arithmetic, if n is very large compared to k, we would get a differenc of 0 and could have large absolute error and relative error. In another situation, consider $x = 0.3721478693$ and $y = 0.3720230572$. We have $x - y = 0.0001248121$. Writing this in floating point arithmetic, $fl(x) = 0.37215$, $fl(y) = 0.37202$ and $fl(x) - fl(y) = 0.00013$, giving large relative error but also showing that we lose decimal places of accuracy in the difference, which we would write as 0.13000×10^{-3}. These are examples to display the error and loss of significance when subtracting nearbly values in floating point arithmetic. In general, we have

Theorem on Loss of Precision: If x and y are positive normalized floating-point binary machine numbers so $x > y$ and

$$2^{-q} \leq 1 - \frac{y}{x} \leq 2^{-p}$$

then at most q and at least p significant binary bits are lost in the subtraction $x - y$.

Proof: Let $x = r \times 2^n$ and $y = s \times 2^m = (s \times 2^{m-n}) \times 2^n$ (where we shift to place x and y on the same exponent, remembering $x > y$). Then,

$$x - y = (r - s \times 2^{m-n}) \times 2^n$$

where $2^{-q} \leq r - s \times 2^{m-n} = r(1 - \frac{y}{x}) \leq 2^{-p}$. Thus, we shift this mantissa left at least p bits and at most q bits, and lost at least p and at most q bits. *QED*

Thus, in cases like computing $\sqrt{1+n^2} - n$ for large n, we should perform algebraic manipulations (multiplying by the conjugate here) to eliminate the subtraction and reduce error. In other cases, we may use techniques like Taylor expansion.

9.3 Stability

[Incomplete]

10 Week 1: Calculus Review (1)

- Add IVT, MVT, Taylor's Theorem
- Add Orders of Convergence, big and little O notation, MVT for integrals, nested multiplication